

Fiche Oral — SAE4D01 DevCloud / Datacenters

Lab Leaf-Spine multi-protocoles sur containerlab — eBGP, iBGP-RR, OSPF, Mixed, EVPN/VXLAN
Benchmark iperf3 comparatif + **monitoring temps réel (Grafana) + preuve ECMP**. Date oral : 2026-06-18

0. Mes 3 atouts différenciants (à placer en intro)

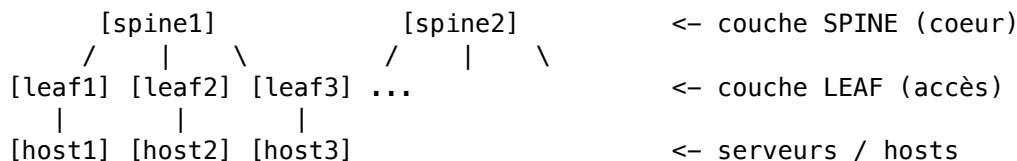
1. **5 topologies** distinctes (eBGP, iBGP-RR, OSPF, mixed, EVPN) — largeur protocolaire.
2. **Benchmark iperf3 chiffré comparatif** (TCP/UDP) — analyse quantitative des perfs.
3. **Observabilité maison** : stack Prometheus + Grafana + **mon propre exporter Python** qui visualise le trafic du fabric EN DIRECT, et **prouve l'ECMP** load-balancing sur les 2 spines.

Phrase d'accroche ECMP : « Tout le monde *configure* l'ECMP, moi je l'ai *mesuré* — et j'ai découvert que sans tuning du hash kernel, il ne se répartit pas. »

1. Le pitch en 30 secondes

« J'ai construit un datacenter virtuel en topologie **leaf-spine** avec **containerlab** sur une VM Proxmox. J'ai déployé **5 topologies** qui implémentent les grands protocoles de routage de datacenter : eBGP, iBGP avec Route Reflector, OSPF, un underlay/overlay mixte, et de l'**EVPN/VXLAN**. J'ai ensuite **benchmarké** les performances réseau de chaque approche avec iperf3 (TCP + UDP), et documenté les bugs rencontrés et leurs corrections. »

2. Architecture (le schéma à savoir dessiner au tableau)



Principe leaf-spine (Clos) : - Chaque **leaf** est connecté à **CHAQUE spine** (full mesh leaf->spine). - Les leaf ne se parlent **jamais** directement, les spine non plus. - N'importe quel host -> host = **toujours 2 sauts** (leaf -> spine -> leaf) = latence prévisible. - Scalabilité horizontale : on ajoute des leaf sans toucher l'existant. ECMP = load-balancing sur les 2 spines.

Pourquoi leaf-spine vs 3-tiers classique (access/distrib/core) ? - Trafic datacenter moderne = surtout **est-ouest** (serveur->serveur), pas nord-sud. - 3-tiers optimisé nord-sud + Spanning-Tree bloque des liens. Leaf-spine = tous les liens actifs (ECMP).

3. Plan d'adressage (commun à toutes les topos)

Liens underlay en /31 (2 IP utilisables, économie d'adresses point-à-point) :

Lien	Subnet
spine1-leaf1	10.0.1.0/31
spine1-leaf2	10.0.1.2/31
spine1-leaf3	10.0.1.4/31
spine2-leaf1	10.0.1.6/31

Lien	Subnet
spine2-leaf2	10.0.1.8/31
spine2-leaf3	10.0.1.10/31

Loopbacks (identité stable du routeur, /32) : spine1=10.255.0.1 · spine2=10.255.0.2 · leaf1=10.255.0.11 · leaf2=10.255.0.12 · leaf3=10.255.0.13

Hosts : routing -> 192.168.1-3.0/24 · EVPN -> 10.10.10.2/3/4 (même L2 étendu en VXLAN)

Pourquoi /31 ? Un lien point-à-point n'a que 2 extrémités. /31 (RFC 3021) donne exactement 2 IP, zéro gaspillage vs /30 qui en gâche 2 (réseau + broadcast). **Pourquoi des loopbacks ?** IP toujours up tant que le routeur vit, indépendante d'une interface physique. Sert d'ID BGP/OSPF et de VTEP en EVPN.

4. Les 5 topologies — ce qu'il faut dire sur chacune

topo-ebgp — la baseline

- **eBGP** : un AS différent par routeur. spine1=65081, spine2=65082, leaf1=65083, leaf2=65084, leaf3=65085.
- Sessions BGP montées sur les IP des liens directs.
- **Pourquoi eBGP en datacenter ?** Simple, scalable, pas de full-mesh requis, propagation naturelle des routes via AS-path. Modèle « BGP-only fabric » (RFC 7938).
- AS-path empêche les boucles automatiquement.

topo-ibgp-rr — iBGP + Route Reflector

- **Un seul AS** : 65080. Tous les routeurs dans le même AS.
- Problème iBGP : règle full-mesh obligatoire (chaque routeur peer avec tous) -> $N \times (N-1) / 2$ sessions, ne scale pas.
- **Solution : Route Reflector**. Les **spines = RR**, les leaf = clients. Le RR ré-annonce (« reflète ») les routes apprises d'un client vers les autres. Plus besoin de full-mesh.
- **Bug clé** -> next-hop-self force (voir section 6).
- ECMP : routes apprises via les 2 spines = 2 chemins égaux.

topo-ospf — OSPF pur

- **OSPF area 0** (single area, backbone). Protocole à état de liens (link-state), calcul Dijkstra/SPF.
- Tous les liens underlay en ip ospf network point-to-point.
- **Bug clé** -> mode broadcast bloque sur /31 (voir section 6).
- Convergence rapide, mais flooding des LSA -> moins scalable que BGP sur très gros fabric.

topo-mixed — underlay OSPF + overlay iBGP-RR

- **OSPF** porte la connectivité underlay (les loopbacks se voient entre eux).
- **iBGP** (AS 65080, spines=RR) monte les sessions **via les loopbacks** (update-source lo), pas via les IP de lien.
- C'est le **modèle réel des datacenters modernes** : underlay simple (IGP) + overlay (BGP) découplés.
- Les loopbacks étant joignables par OSPF, la session BGP survit à la perte d'un lien physique (reroute par OSPF). Robustesse.
- Top du benchmark le 17/06 (10.48 G TCP), 3e le 19/06 (10.11 G) — tous les protocoles sont dans le bruit de mesure (cf. section 5).

topo-evpn — EVPN/VXLAN (la plus avancée)

- **Underlay** : OSPF. **Overlay** : BGP **EVPN** (address-family l2vpn evpn), AS 65080.
- **VXLAN** : encapsulation L2-over-L3. VNI 100. Les hosts (10.10.10.2/3/4) sont sur le **même réseau L2 logique** alors qu'ils sont sur des leaf physiquement séparés.
- **VTEP** = VXLAN Tunnel Endpoint = la loopback de chaque leaf. C'est là que le trafic est encapsulé/décapsulé.
- **EVPN** = le control-plane : distribue les MAC/IP des hosts via BGP (Type 2 routes) au lieu du flood-and-learn classique.
- **Cas d'usage** : multi-tenant, mobilité de VM (live migration sans changer d'IP), stretch L2 entre racks.

5. Résultats benchmark iperf3 — LES 5 TOPOS (run 2026-06-19)

Topo	TCP Gbit/s	UDP Gbit/s
topo-ebgp	11.07	8.01
topo-ibgp-rr	10.92	7.61
topo-mixed	10.11	8.02
topo-evpn	9.76	10.0
topo-ospf	9.52	8.34

Bench lancé en 100% containerlab via /root/benchmark-all.sh (deploy -> converge 35s -> TCP+UDP 30s -> destroy, par topo). EVPN inclus après fix MTU (bug 5).

Lecture / ce qu'on en conclut : - **eBGP le plus rapide ce run (11.07 TCP)**, mais tous dans un mouchoir 9.5–11 Gbit/s. **Le protocole de routage n'est PAS le goulot** : une fois la route dans la FIB, c'est le data-plane (forwarding kernel/veth) qui plafonne, pas BGP/OSPF. - **Variance run-à-run** : le 17/06 mixed était devant (10.48), le 19/06 c'est eBGP (11.07). Écarts < 1 Gbit/s = **bruit de mesure** en environnement virtuel. Conclusion honnête : pas de gagnant net en débit pur, la vraie différence est ailleurs (convergence, scalabilité, fonctionnalités). - **EVPN un cran en dessous (9.76)** : normal, le VXLAN ajoute ~50 octets d'encapsulation par paquet -> overhead réel. - **UDP** : iperf3 envoie à débit forcé (-b 10G), on mesure le débit atteignable avant pertes. EVPN UDP « 10.0 » = débit d'émission (datagrammes petits, 0% perte). - **Méthode** : 1 run par topo = indicatif, pas statistiquement robuste. Pour rigueur il faudrait moyenner plusieurs runs (limite que j'assume).

6. Bugs rencontrés + fixes (LA partie qui impressionne le jury)

Bug 1 — iBGP RR : next-hop non modifié

- **Symptôme** : routes réfléchies par le RR ont un next-hop injoignable, pas installées.
- **Cause** : en iBGP, un RR ne change PAS le next-hop par défaut (préserve l'attribut). next-hop-self seul ne suffit pas dans FRR pour les routes **réfléchies**.
- **Fix** : next-hop-self force sur chaque neighbor client du RR. Le force force la réécriture même sur routes reflétées.

Bug 2 — OSPF bloqué en 2-Way/DROther sur liens /31

- **Symptôme** : adjacence OSPF coince à 2-Way/DROther, jamais Full.
- **Cause** : OSPF en mode **broadcast** (défaut sur ethernet) tente d'élire un DR/BDR. Sur un /31 point-à-point, élection inutile et bloquante.
- **Fix** : ip ospf network point-to-point sur chaque interface underlay -> pas de DR, adjacence directe Full.

Bug 3 — VXLAN : timing de l'interface

- **Symptôme** : interface vxlan100 échoue à la création.

- **Cause** : la commande `ip link add vxlan100 ... local <IP>` a besoin que l'IP de loopback **existe déjà** avant.
- **Fix** : dans l'exec du container leaf, ajouter `ip addr add 10.255.0.X/32 dev lo` **AVANT** la création de l'interface vxlan.

Bug 5 — EVPN : TCP à 0 Gbit/s alors que ping OK (MTU VXLAN) □ trouvé pendant le bench

- **Symptôme** : sur topo-evpn, le ping host-à-host passe (0% perte), l'UDP iperf3 atteint 10 G, mais le **TCP iperf3 reste bloqué à 0 Gbit/s**.
- **Diagnostic** : VXLAN ajoute ~50 octets d'encapsulation. L'interface `vxlan100` avait été créée avec la **MTU par défaut 1500**, alors que les hosts étaient en jumbo (MTU 9500). Les petits paquets (ping, handshake TCP, datagrammes UDP par défaut) passent, mais dès que TCP envoie des **segments pleins**, ils dépassent la MTU effective du tunnel -> ils sont silencieusement dropped. Le PMTUD (Path MTU Discovery) ne récupère pas car les ICMP « fragmentation needed » ne reviennent pas à travers le VXLAN -> **blackhole TCP**.
- **Fix** : monter la MTU de `vxlan100` (et du bridge) à 9000 sur chaque leaf, host à 8950. Résultat : **TCP passe de 0 à 9.76 Gbit/s**.
- **Argument** : « le piège classique du VXLAN — ça ping, donc on croit que ça marche, mais le TCP blackhole sur la MTU. Un bug qu'on ne voit qu'en *mesurant* le débit, pas en pingant. »

Bug 4 — ECMP ne se répartit pas (hash kernel L3) □ LE plus fort

- **Symptôme** : 2 chemins ECMP présents dans la FIB (via spine1 ET spine2), mais 100% du trafic passe par UN seul spine, même avec 8 flux iperf3 parallèles.
- **Diagnostic** : la route BGP a un next-hop unique (loopback de leaf3), résolu par OSPF en 2 chemins underlay (les 2 marqués * dans `show ip route`). L'ECMP **existe** donc bien. Le problème est le **hash de répartition du kernel Linux** : par défaut `fib_multipath_hash_policy=0` = hash **L3** (IP source + destination seulement). Comme tous les flux ont la même paire host1->host3, ils hashent tous vers le même spine. Augmenter -P ne change rien (même IP src/dst).
- **Fix** : `sysctl -w net.ipv4.fib_multipath_hash_policy=1` sur chaque leaf -> hash **L4** (inclut les ports TCP/UDP). Les flux à ports sources différents se répartissent alors sur les 2 spines.
- **Preuve mesurée** : avant fix -> spine1 0% / spine2 100%. Après fix -> spine1 62% / spine2 38% (les 2 spines actifs). Visible en direct dans Grafana.
- **Argument** : « configurer maximum-paths ne suffit pas, encore faut-il que le plan de forwarding hashe sur assez d'entropie. »

6bis. Monitoring & observabilité (ce que je montre à l'écran) □

Stack déployée sur VM220 (/root/monitoring/, docker compose) : - **Prometheus** (:9090) — base de métriques time-series, scrape toutes les 2s. - **Grafana** (:3000) — dashboards. Accès : `http://10.202.8.220:3000`. - **node-exporter** — métriques de l'hôte (RAM/CPU VM220). - **clab_exporter.py** — mon exporter Python maison (~120 lignes).

Pourquoi un exporter maison ? cAdvisor (l'outil standard) **échoue** sur ce setup : bug de mapping overlay2/cgroup v2 (failed to identify the read-write layer ID) -> il ne voit pas les conteneurs containerlab et ne donne aucune métrique réseau par nœud. J'ai donc écrit un exporter qui entre dans la **netns** de chaque conteneur via nsenter et lit /proc/net/dev -> expose le débit RX/TX par conteneur/interface au format Prometheus sur :9101.

Dashboard « Fabric Leaf-Spine Live » (sélecteur de topo en haut) : - Débit TX par nœud -> on **voit le flux iperf3 traverser** host1 -> leaf1 -> spine -> leaf3 -> host3. - **Panel ECMP** : débit RX par spine -> prouve la répartition sur les 2 spines. - Camembert part de trafic par spine, paquets/s, RAM/CPU hôte.

Démo live : `bash /root/monitoring/demo-ecmp.sh` mixed lance 8 flux 60s, à regarder bouger dans Grafana.

Face à Valentin : lui a Grafana + Prometheus + sFlow, mais **branchés sans analyse de perf**. Moi je relie le monitoring à un **benchmark chiffré** et à une **découverte technique** (le hash ECMP). Monitoring + mesure + diagnostic, pas juste « j'ai installé Grafana ».

7. La stack technique (savoir justifier les choix)

- **Proxmox** : hyperviseur, héberge la VM du lab. (VM220 = leaf-spine-lab1, 15 GB RAM, 8 cores, Debian.)
 - **containerlab** (v0.76.1) : orchestre des topologies réseau en **conteneurs Docker** déclarées en YAML. Léger vs GNS3/EVE-NG (pas de VM par nœud).
 - **FRR** (FRRouting v10.6.1, image quay.io/frrouting/frr) : suite de routage open-source (descendant de Quagga). Fournit bgpd, ospfd, zebra. C'est le « Cisco IOS open-source ».
 - **vytysh** : le CLI unifié de FRR (style Cisco), `show bgp summary`, etc.
 - **Alpine + iperf3** : hosts légers pour générer le trafic de test.
 - **Pourquoi conteneurs vs VM ?** Démarrage en secondes, faible empreinte -> 5 topos sur une seule VM. Idéal pour du lab réseau reproductible (infra-as-code en YAML).
-

8. Commandes de démo (si on demande de montrer en live)

Déployer / détruire une topo

```
cd /root/topo-ebgp && containerlab deploy -t topology.clab.yml
containerlab destroy -t topology.clab.yml --cleanup
containerlab inspect --all
```

BGP

```
docker exec clab-topo-ebgp-spine1 vtysh -c "show bgp summary"
docker exec clab-topo-ebgp-leaf1 vtysh -c "show ip route bgp"
```

OSPF

```
docker exec clab-topo-ospf-spine1 vtysh -c "show ip ospf neighbor"
```

EVPN / VXLAN

```
docker exec clab-topo-evpn-spine1 vtysh -c "show bgp l2vpn evpn summary"
docker exec clab-topo-evpn-leaf1 vtysh -c "show evpn vni"
docker exec clab-topo-evpn-leaf1 vtysh -c "show evpn mac vni 100"
```

Test perf

```
docker exec -d clab-topo-ebgp-host3 iperf3 -s
docker exec clab-topo-ebgp-host1 iperf3 -c 192.168.3.2 -t 30 -J # TCP
docker exec clab-topo-ebgp-host1 iperf3 -c 192.168.3.2 -t 30 -u -b 10G # UDP
```

--- MONITORING / ECMP (la partie qui me distingue) ---

Grafana : <http://10.202.8.220:3000> (dashboard "Fabric Leaf-Spine Live")

```
docker ps --filter name=mon- # stack monitoring
systemctl status clab-exporter # mon exporter Python
curl -s localhost:9101/metrics | grep clab_net # métriques brutes
```

Démo ECMP live (active hash L4 + 8 flux 60s, à regarder dans Grafana) :

```
bash /root/monitoring/demo-ecmp.sh mixed
```

*# Preuve ECMP en CLI : route avec 2 next-hops (les 2 spines, marqués *)*

```
docker exec clab-topo-mixed-leaf1 vtysh -c "show ip route 192.168.3.0/24"
```

9. Questions pièges probables + réponses

Q : Différence eBGP / iBGP ? > eBGP = entre AS différents, modifie le next-hop et l'AS-path, TTL 1 par défaut. iBGP = même AS, ne modifie pas l'AS-path, exige full-mesh ou un Route Reflector. iBGP préserve les attributs, eBGP les réécrit.

Q : À quoi sert un Route Reflector ? > Casser la contrainte de full-mesh iBGP. Au lieu de $N \times (N-1) / 2$ sessions, les clients peer juste avec le(s) RR qui ré-annoncent les routes. Réduit drastiquement le nombre de sessions. Ici les spines sont RR.

Q : OSPF vs BGP, lequel choisir en datacenter ? > OSPF (link-state) converge vite, simple sur petit fabric mais flood les LSA -> limite de scalabilité. BGP scale à des milliers de nœuds, plus de contrôle de policy, c'est le standard des grands datacenters (RFC 7938). En pratique : underlay simple + overlay BGP (mon topo-mixed/evpn).

Q : Ton ECMP, comment tu prouves qu'il marche vraiment ? > Deux niveaux. (1) Plan de contrôle : `show ip route` montre 2 next-hops marqués * (via spine1 et spine2). (2) Plan de données : je mesure le débit RX de chaque spine dans Grafana pendant un iperf3. J'ai découvert qu'un flux unique ne se répartit pas (1 seul chemin par flux, hash sur le 5-tuple), et que même 8 flux restaient sur un spine tant que le kernel hashait en L3 (IP src/dst identiques). En passant `fib_multipath_hash_policy=1` (hash L4 avec les ports), la charge s'est répartie sur les 2 spines : passée de 100/0 à 62/38.

Q : Pourquoi un exporter maison plutôt que cAdvisor ? > J'ai d'abord déployé cAdvisor (le standard) mais il échoue sur ce setup : bug overlay2/cgroup v2, il ne mappe pas les conteneurs containerlab -> zéro métrique réseau par nœud. J'ai écrit un exporter Python qui entre dans la netns de chaque conteneur (nsenter) et lit /proc/net/dev, exposé au format Prometheus. ~120 lignes, fiable, et je maîtrise exactement ce qui est mesuré.

Q : Pourquoi FRR et pas Arista cEOS (comme ton binôme) ? > FRR est open-source, gratuit, léger (conteneur de quelques Mo) -> je tiens 5 topologies sur une seule VM. cEOS est un vrai NOS d'entreprise, plus réaliste mais plus lourd et sous licence/image privée. FRR me suffit pour démontrer les protocoles, et c'est exactement ce qui tourne dans beaucoup de fabrics réels (Cumulus/SONiC s'appuient dessus).

Q : C'est quoi VXLAN exactement ? > Encapsulation d'une trame Ethernet (L2) dans un paquet UDP/IP (L3). Permet d'étendre un domaine L2 par-dessus un réseau L3 routé. Identifié par un VNI (24 bits -> 16M segments vs 4096 VLAN). Résout la limite des VLAN et permet la mobilité L2 entre racks.

Q : VXLAN vs EVPN, c'est pareil ? > Non. VXLAN = le **data-plane** (l'encapsulation). EVPN = le **control-plane** (comment on apprend et distribue les MAC/IP, via BGP). Sans EVPN, VXLAN utilise du flood-and-learn multicast, peu scalable. EVPN remplace ça par du BGP. Les deux ensemble = standard datacenter.

Q : C'est quoi un VTEP ? > VXLAN Tunnel Endpoint. Le point où le trafic entre/sort du tunnel VXLAN (encapsulation/décapsulation). Ici = la loopback de chaque leaf.

Q : C'est quoi ECMP ? > Equal-Cost Multi-Path. Quand plusieurs chemins de coût égal existent (via spine1 ET spine2), le routeur load-balance le trafic dessus. Tous les liens leaf-spine sont actifs, pas de Spanning-Tree qui en bloque.

Q : Pourquoi /31 et pas /30 sur les liens ? > Un lien point-à-point a 2 extrémités. /31 (RFC 3021) = exactement 2 IP. /30 gâche 2 adresses (réseau+broadcast). Économie sur un grand fabric.

Q : Comment tu as validé que ça marche ? > Trois niveaux : (1) état du protocole — `show bgp summary` / `show ip ospf neighbor` en Established/Full ; (2) table de routage — routes apprises présentes dans la FIB ; (3) connectivité réelle — `ping host-à-host` + benchmark iperf3 du débit.

Q : Limites de ton lab / ce que tu améliorerais ? > Virtuel (veth/kernel) -> débits pas représentatifs de hardware réel (pas d'ASIC). Single VM -> pas de test de panne physique inter-rack. Prochaines étapes : EVPN Type 5 (routage L3 inter-VNI via symmetric IRB), étendre sur VM221 (lab multi-VM), versionner les configs en Git.

Q : Que se passe-t-il si un spine tombe ? > ECMP bascule tout le trafic sur le spine restant (convergence du protocole). En topo-mixed, comme BGP source ses sessions sur les loopbacks joignables via OSPF, la session BGP peut même survivre à la perte d'un lien si un autre chemin existe.

10. Glossaire express (à relire 5 min avant)

Terme	Définition courte
AS	Autonomous System, domaine de routage identifié par un numéro
AS-path	Liste des AS traversés, anti-boucle BGP
FIB / RIB	Forwarding/Routing Information Base (table forwarding / table routage)
LSA	Link-State Advertisement, l'unité d'info OSPF
SPF	Shortest Path First, l'algo (Dijkstra) d'OSPF
Loopback	Interface virtuelle toujours up, ID du routeur
Underlay	Réseau physique/IP de base qui transporte
Overlay	Réseau logique (VXLAN) par-dessus l'underlay
VNI	VXLAN Network Identifier (24 bits)
VTEP	VXLAN Tunnel Endpoint
IRB	Integrated Routing and Bridging (routage inter-VNI)
RR	Route Reflector
ECMP	Equal-Cost Multi-Path
Clos	Topologie réseau non-bloquante (leaf-spine en est dérivée)

““